

1.1.1 Introduction

Computer Vision (CV): 用计算机理解 image/video 的 meaning, 即 Marr: know what is where by looking. 图像来自 **sensing device**, CV 是 **graphics inverse problem**: graphics model $\rightarrow$ image, vision image $\rightarrow$ world/model. **Understand image**: objects/scene, 3D/material properties, actions, relations, 可回答关于图像的问题。**Applications**: measurement (**shape/motion capture**, **3D urban modeling**, **camera ruler**, **photo tourism**), recognition (**face/OCR**/fine-grained/action), reconstruction, robotics, **AR/VR**, medical, autonomous driving. **Core difficulties**: inverse problem is ill-posed; projection loses depth; appearance changes with illumination/viewpoint/occlusion/deformation/background; semantic gap from pixels to meaning. **Task modes**: measurement (shape/motion/3D), recognition (semantic content), manipulation/generation (**impainting**, **style/domain transfer**, **deepfake**). Progress is dataset-driven: larger/richer data enables captioning, interaction, egocentric/video and multimodal tasks.

2.1.2 Image Basics & Filters

2.1 Digital Image

Digital camera: sensor array converts photons to electrons. Image formation includes **sampling** continuous image plane and **quantizing** intensity. **Grayscale image** $I: \mathbb{R}^2 \rightarrow \mathbb{R}$; **digital image** $I \in \mathbb{R}^{H \times W \times C}$, usually **uint8** $[0, 255]$. **Resolution** is spatial sampling density. Low sampling causes **aliasing**: quantization causes intensity error. **Image as function**: transformation can be **pointwise/intensity**, e.g. $g(x, y) = f(x, y) + 50$, **geometric** $g(x, y) = f(x, -y)$, **threshold/mask** $g = f \cdot 1[f < 250]$. **Color**: perceived property from light/surface/visual system. **RGB** is linear primary weights; **HSV** separates hue/saturation/value, useful when color appearance and brightness need separate handling. **Histogram** counts intensities/color bins; useful for contrast, thresholding, matching, normalization. **Noise**: **salt-and-pepper** = random black/white spikes; **impulse** = sparse white spikes; **Gaussian** = iid normal intensity perturbation. **Mean/Gaussian smoothing** assumes neighboring pixels are similar and noise is independent, so averaging suppresses noise but blurs edges.

2.2 Correlation, Convolution, Filtering

Correlation: $G = H \otimes F$, $G[i, j] = \sum_{u,v} H[u, v]F[i + u, j + v]$. **Convolution**: kernel flipped, $(H * F)[i, j] = \sum_{u,v} H[u, v]F[i - u, j - v]$. For symmetric kernels, correlation = convolution. Boundary: zero fill, replicate border, mirror reflect, mirror across edge. **Linear filters**: shifting, blur, sharpen, edge-like high-pass. **Box filter** averages but creates blocky response. **Gaussian filter** $G_G(x, y) = \frac{1}{2\pi\sigma^2} \exp(-(x^2 + y^2)/(2\sigma^2))$; smoothing suppresses high frequency/noise; larger σ means stronger blur. Median filter is nonlinear, robust to salt-and-pepper noise. **Separable filter**: $K = wv^T$, so 2D convolution becomes horizontal then vertical; cost $O(k^2 HW) \rightarrow O(2kHW)$. Gaussian is separable. **Sharpening** can be viewed as $I + \alpha(I - \text{blur}(I))$. **Kernel facts**: smoothing kernels usually positive and sum to 1, preserving constant images. Output shape choices: **full** keeps all overlaps; **same** keeps input size; **valid** only positions where kernel fully fits. **Convolution properties**: shift-invariant, commutative, associative, distributive, identity impulse δ , and $\mathcal{F}(f * g) = \mathcal{F}(f)\mathcal{F}(g)$; Gaussian blur is low-pass. **Template matching**: filters can be matched templates; use **normalized cross-correlation** to reduce brightness bias. **Hybrid image**: low-pass one image + high-pass another, perceived content changes with viewing distance/scale. Separable iff $\text{rank}(K) = 1$; **SVD** gives best rank-1 approximation for nearly separable filters.

3.1.3 Camera Models

3.1 Perspective Projection

Camera center C , **optical axis** Z , image plane distance f , **principal point** $p = (p_x, p_y)$. For camera coordinate $Q = (X, Y, Z)^T$: $x = \frac{fX}{Z} + p_x$, $y = \frac{fY}{Z} + p_y$. Depth ambiguity: one pixel corresponds to a whole ray. Larger **focal length** $\Rightarrow$ narrower **field of view**. **Pinhole/lens**: tiny aperture gives sharp image but little light and diffraction; large aperture gives more light but blur; lens focuses rays from one depth plane onto sensor. Projection loses depth, metric distances and angles; it preserves incidence of points/lines, but objects through camera center may degenerate. **Homogeneous coordinates**: $(x, y, 1)^T \sim (sx, sy, s)^T$. Line $l = (a, b, c)^T$ means $l^T x = 0$; line through two points $l = x_1 \times x_2$. Intrinsic: $K = \begin{bmatrix} f_x & 0 & p_x \\ 0 & f_y & p_y \\ 0 & 0 & 1 \end{bmatrix}$, $\tilde{x} \sim KX_c$. Square/no skew: $s = 0, f_x = f_y = f$. Full projection: $X_c = RX_w + t = R(X_w - C)$, $P = K[R \mid t]$, $\tilde{x} \sim PX_w$. Projection matrix decomposition: $P = K\Pi_0 \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} I & T \\ 0 & 1 \end{bmatrix}$, $K = \begin{bmatrix} f & 0 & p_x \\ 0 & f & p_y \\ 0 & 0 & 1 \end{bmatrix}$, $\Pi_0 = [I_3 \mid 0]$. After $\tilde{x} = PX$, divide by third coordinate.

3.2 Projective Properties

Points map to points; 3D lines generally map to image lines; parallel 3D lines may meet at vanishing point. For line $X(t) = V + tD$, as $t \rightarrow \infty$, projected point depends only on direction D : $\tilde{x}_v \sim KD$, $D \sim K^{-1}\tilde{x}_v$. **Vanishing points** of directions in a plane lie on the **vanishing line**; ground-plane vanishing line is **horizon**. Parallel planes share horizon line. **Orthographic projection** drops depth: $(X, Y, Z, 1)^T \rightarrow (X, Y, 1)^T$ with no perspective division; good when depth variation is small relative to distance. **VP caveats**: lines parallel to image plane meet at infinity; 2D image intersection does not imply 3D parallel lines. **Calibration**: known **3D calibration pattern** + detected 2D corners estimates K and extrinsics; three orthogonal vanishing points can calibrate **Manhattan scenes**. **Single-view metrology**: with flat ground and level camera, horizon height can reveal camera height on vertical objects; with known K and ground plane, backproject image rays to plane and place CAD/measure scene.

4.1.4 Stereo & Epipolar Geometry

4.1 Parallel Calibrated Stereo

Two views turn a left pixel ray into an **epipolar line** in right image; matching + **triangulation** recovers 3D. For rectified parallel cameras with **baseline** B , focal f : $y_l = y_r$, $d = x_l - x_r$, $Z = \frac{fB}{d}$. Closer objects have larger **disparity**. Full 3D from left image: $X = \frac{(x_l - p_x)Z}{f}$, $Y = \frac{(y_l - p_y)Z}{f}$.

4.2 General Epipolar Geometry

Epipole e_r : left camera center projected into right image. **Epipolar plane** (C_l, C_r, P) intersects images in paired epipolar lines; matching search becomes 1D. **Fundamental matrix** maps point to line: $l_r = Fp_l$, $p_r^T Fp_l = 0$. Construction: $F = [e_r]_{\times} H_{\pi}$, where H_{π} is plane-induced homography and $[e]_{\times} = \begin{bmatrix} 0 & -e_3 & e_2 \\ e_3 & 0 & -e_1 \\ -e_2 & e_1 & 0 \end{bmatrix}$. Each correspondence gives linear equation in F entries: $x_r x_l [f_{11} + x_r y_l [f_{12} + x_r f_{13} + y_r x_l [f_{21} + y_r y_l [f_{22} + y_r f_{23} + x_l f_{31} + y_l [f_{32} + f_{33} = 0$. F has scale ambiguity and rank constraint; use multiple matches + least squares/RANSAC.

Rectification warps images so epipolar lines become horizontal. From F one projective camera choice: $P_l = [I \mid 0]$, $P_r = [[e_r]_{\times} F \mid e_r]$. **F facts**: F has scale ambiguity + rank-2 constraint, so 7 DoF; $e_r^T F = 0$ and $F e_l = 0$. Need $n \geq 7$ matches in principle, more with RANSAC in practice. **SfM: multi-view stereo** jointly estimates cameras and 3D points by minimizing **reprojection error** $\sum_{ij} \|x_{ij} - \pi(P_i X_j)\|^2$; solved by **bundle adjustment**. **Structured light**: projector emits known pattern to simplify correspondence; **Kinect** uses IR pattern/depth sensing.

5.1.5 Keypoints, Features, Matching

Local feature goal: match corresponding image points from same world point. Recall needs invariance to rotation/scale/affine/illumination; precision needs contrast/distinctiveness and locality. Raw patch descriptor is sensitive to shifts/rotations. **Detector desiderata**: salient + repeatable under viewpoint/illumination. Matching all pixels increases false positives unless descriptor is extremely strong.

5.1 SIFT & Keypoint Selection

SIFT Descriptor: **Scale-Invariant Feature Transform**. **Gaussian pyramid/scale-space** enables scale invariance; illumination robustness from **normalized gradients**; rotation invariance from **dominant orientation**. Descriptor: rotate local grid to dominant gradient, split into 4×4 cells, each with 8-bin gradient histogram $\Rightarrow$ 128D descriptor. **SIFT matching**: **nearest neighbor** in Euclidean descriptor space; **threshold ratio** of nearest to 2nd nearest descriptor. Low ratio = good match; high ratio = ambiguous. **Corners**: intensity changes in all directions. **Harris** uses local second-moment matrix: $\Delta^2 \approx [\Delta x \ \Delta y] \begin{bmatrix} D_x^2 & D_x D_y \\ D_x D_y & D_y^2 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$. Large eigenvalues in both directions $\Rightarrow$ corner; one large one small $\Rightarrow$ edge; both small $\Rightarrow$ flat. **Harris response** often $R = \det M - k(\text{tr} M)^2$. Scale-invariant interest points: **Laplacian of Gaussian** $\nabla^2 G_{\sigma} * I$ approximated by **Difference of Gaussian** $(G_{k\sigma} - G_{\sigma}) * I \approx (k - 1)\sigma \partial_{\sigma} (G_{\sigma} * I)$.

5.2 Homography & RANSAC

Transformations: translation/rotation/similarity/affine/projective. Affine preserves parallel lines; projective/homography may send parallel lines to vanishing point. Planar scene or pure camera rotation gives **homography**: $\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} \sim H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$, $H = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \sim \lambda H$, $T(t) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$, $A = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix}$. Translation has identity upper-left 2×2 and right column (t_x, t_y) ; affine uses arbitrary upper-left 2×2 plus translation column. H has 8 DoF up to scale; each point pair gives 2 equations, so solving needs ≥ 4 pairs; over-constrained solve least squares. **Image mosaic**: reproject images from same center of projection into a shared synthetic wide-angle camera. **2D transform hierarchy**: translation 2 DoF; similarity 4 DoF (translation+rotation+uniform scale); affine 6 DoF, preserves parallel lines; homography/projective 8 DoF, preserves straight lines but not parallelism. **Warping**: **forward warp** can create holes/non-integer hits; **inverse sampling** with interpolation is usually cleaner. **Mosaic pipeline**: detect/match features $\rightarrow$ estimate H with RANSAC $\rightarrow$ warp to common frame $\rightarrow$ blend.

RANSAC: robust fitting under outliers; least squares is fragile because squared residuals let bad matches dominate. **Inlier**: residual below threshold; **outlier**: inconsistent match/noise/other structure; **consensus set**: inliers supporting a model. Loop: sample minimal seed, fit model, count inliers under threshold t , keep largest consensus, refit on all inliers. For homography seed size $s = 4$; residual after homogeneous division: $p'_i = \text{dehom}(H p_i)$, $e_i = \|p'_i - p'_i\|_2$, $e_i < t \Rightarrow$ inlier. Trial count for success probability P , inlier ratio w , sample size s : $N = \frac{\log(1-P)}{\log(1-w^s)}$, $1 - P = (1 - w^s)^N$. Tune sample size s , threshold t , success probability P , max trials N , min consensus size. Pros: simple/general, often works well; cons: poor with low inlier ratio (w^s tiny), bad thresholds, degenerate/minimal samples.

6.1.6 Vision Neural Networks

6.1 CNN Basics

CNN biases: local connectivity, weight sharing, spatial structure. **Conv filter** $F \times F \times C$ slides over input; output depth = number of filters K . $W_2 = \frac{W_1 - F + 2P}{S} + 1$, $H_2 = \frac{H_1 - F + 2P}{S} + 1$, $\#0 = F^2 CK + K$. Pooling: per-channel downsample, no parameters; **max pooling** $W_2 = (W_1 - F)/S + 1$. **1×1 conv** mixes channels only, useful for projection/bottleneck. **ReLU** $f(x) = \max(0, x)$: efficient, non-saturating positive side, but dead ReLU/output not zero-centered. **FC vs Conv**: **FC** flattens HWC into a vector; params for M outputs are $HWC \cdot M + M$, no spatial sharing. Conv keeps the grid, uses local receptive

fields and shared weights. **Sigmoid/tanh** / **LeakyReLU/ELU**

Initialization/normalization: constant init breaks symmetry poorly; small random can vanish/explode. Xavier std $1/\sqrt{D_{in}}$. LayerNorm: $\mu = \frac{1}{d} \sum_j x_j$, $\sigma^2 = \frac{1}{d} \sum_j (x_j - \mu)^2$, $y_j = \gamma \frac{x_j - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$. **Regularization/generalization**: early stopping, data augmentation, dropout, stochastic depth, random crop, Mixup. **Regularization view**: always keep best validation snapshot; augmentation/dropout/stochastic depth can be read as training with injected noise, while test-time prediction marginalizes/averages over the noise.

6.2 Architectures

AlexNet: ReLU, data augmentation, GPU split, ImageNet 2012. **ResNet** solves deep optimization by residual block $H(x) = F(x) + x$; if $F = 0$ identity is easy. **Bottleneck**: 1×1 reduce, 3×3 , 1×1 expand. **DenseNet**: each layer connects to all later layers, feature reuse, gradient flow. **Depthwise separable conv** decomposes standard conv into depthwise spatial + pointwise channel: $K^2 C^2 \rightarrow K^2 C + C^2$. **ConvNeXt**: Transformer ideas in CNN: patchify stem, depthwise conv, inverted bottleneck, GeLU, BatchNorm $\rightarrow$ LayerNorm, fewer norm layers, separate downsampling. **U-Net**: encoder-decoder with long skip connections for localization; descendants in segmentation, diffusion, point cloud. **ResNet head**: drops large FC stack; last conv $\rightarrow$ global average pooling $\rightarrow$ class FC, reducing params and supporting variable spatial input. **ConvNeXt V2**: not just architecture; ConvNeXt blocks are co-designed with MAE-style/self-supervised training.

6.3 Attention, Transformer, ViT

Attention with query Q , data X : $K = XW_K$, $V = XW_V$, $E = \frac{QK^T}{\sqrt{D}}$, $A = \text{softmax}(E)$, $Y = AV$. **Self-attention**: $Q = XW_Q$, $K = XW_K$, $V = XW_V$. **Masked attention** sets illegal future logits to $-\infty$. **Multi-head**: H heads, $D_H = D/H$, concatenate then output projection. Complexity/memory $O(N^2)$ due to attention matrix; **FlashAttention** avoids materializing full $N \times N$ matrix. **Transformer block**: self-attention is vector interaction; LayerNorm/MLP are per-tokens; pre-norm makes residual identity easier. **ViT**: split image into patches, flatten, linear project to tokens, add positional embedding, Transformer, pool/[CLS], classify. Example $16 \times 16 \times 3 = 768$ patch vector. ViT weakens CNN translation equivariance but gives global interaction. RMSNorm: $y_i = \frac{x_i}{\sqrt{e + \frac{1}{N} \sum_j x_j^2}} \gamma_i$.

7.1.7 Classification & Representation Learning

7.1 Recognition & Self-Supervision

Recognition tasks: identification, scene/object classification, annotation, detection, segmentation, pose, attributes, action. Classification challenge: semantic gap, viewpoint, clutter, illumination, occlusion, deformation, intra-class variation. **Supervised pipeline**: collect labels, train classifier, evaluate; expensive labels motivate **self-supervised learning (SSL)**. **Pretext vs downstream**: labels generated from data itself for pretraining; downstream is real labeled task. **Pretext risk**: auto-generated labels may teach task artifacts rather than transferable semantics if the pretext task is too narrow. **MAE**: mask high ratio patches (e.g. 75%), **encoder** sees visible patches only, **decoder** reconstructs pixels at masked patches; keep encoder, discard decoder in downstream tasks. Loss: $\mathcal{L}_{MAE} = \frac{1}{|M|} \sum_{i \in M} \|x_i - \hat{x}_i\|_2^2$. Asymmetric: large encoder, small decoder.

7.2 Contrastive Learning

Goal: positive pairs close, negatives far. **InfoNCE**: $L = -E \log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))}$. **SimCLR**: two augmentations of same image form positive; cosine score $s(u, v) = u^T v / (\|u\| \|v\|)$; encoder $h = f(\tilde{x})$, projection head $z = g(h)$. **NT-Xent** for positive (i, j) : $\ell_{ij} = -\log \frac{\exp(s(z_i, z_j)/\tau)}{\sum_{k \neq i} \exp(s(z_i, z_k)/\tau)}$. Downstream uses h not z . **MoCo**: queue of keys decouples negatives from batch size; key encoder EMA: $\theta_k \leftarrow m\theta_k + (1 - m)\theta_{k'}$. Queue motivation: extra forward passes for many random negatives are expensive; precomputed dataset features quickly become stale as encoder updates; queue reuses recent mini-batch keys as a cheap/stable compromise. **DINO**: student matches teacher outputs; teacher EMA, centering+sharpening avoid collapse: $\theta_k \leftarrow \tau\theta_k + (1 - \tau)\theta_s$. DINO's ViT attention maps often localize object regions without labels. **DINOv2** scales curated data + global DINO + local masked/iBOT, strong patch features. **SSL eval**: pretrain encoder, discard projection head, then freeze+linear probe or fine-tune on downstream labels.

7.3 JEPA

JEPA aligns embeddings $s_x = \text{Enc}_x(x)$, $s_y = \text{Enc}_y(y)$. **JEPA** predicts target representation: $\hat{s}_y = \text{Pred}(\text{Enc}_x(x), z)$, $\mathcal{L} = \|s_y - \hat{s}_y\|^2$. Not pixel reconstruction; predicts abstract representation and avoids irrelevant texture. Because Enc_y is not invertible, prediction in representation space need not reconstruct full y . **H-JEPA** passes latents hierarchically to predict more abstract/longer-term structure. **Collapse**: all inputs map to constant; **SIGReg** regularizes latent statistics to avoid collapse.

7.4 Vision-Language Representation

CLIP: (image, text) $\rightarrow$ matched pair. Image encoder and text encoder produce embeddings; batch similarity: $l_i = f_{\text{img}}(x_i)$, $t_j = f_{\text{text}}(t_j)$, $S_{ij} = l_i^T t_j$. Diagonal entries are positives, off-diagonal entries are negatives; CLIP trains both image-to-text and text-to-image retrieval. Compared with SimCLR, positives are Internet image-text pairs rather than two augmentations of one image. **Zero-shot classification**: text embeddings act like classifier weights: $\hat{c} = \arg \max_{c \in \mathcal{C}} f_{\text{img}}(x)^T f_{\text{text}}(\text{"a photo of a "})$.

SigLIP: replaces CLIP softmax competition with pairwise sigmoid binary matching, decoupling loss behavior from batch-size-as-class-count: $\mathcal{L}_{\text{SigLIP}} = -\frac{1}{|B|} \sum_{i,j} \log \sigma(z_{ij}(tx_i^T y_j + b))$, $z_{ij} \in \{-1, -1\}$. $z_{ij} = +1$ for matched diagonal pairs and -1 for mismatched pairs; t is learnable temperature, b learnable bias. **CoCa**: CLIP-style contrastive objective + captioning decoder; contrastive loss gives global image-text alignment, captioning loss $p(w_1, \dots, w_T | \text{image})$ gives finer language supervision.

7.5 Vision-Language Models & Data Caveats

LLaVA: vision encoder $\rightarrow$ projector/modality adapter $\rightarrow$ visual tokens in LLM context: image $\xrightarrow{\text{vision enc.}}$ visual features $\xrightarrow{\text{projector}}$ visual tokens, [visual tokens, text tokens] $\xrightarrow{\text{LLM}}$ output text. Two-stage training: image-text alignment trains projector so visual tokens fit LLM space; visual instruction tuning teaches response format and question answering. **Flamingo**: inserts gated cross-attention into the language model; visual encoder/downsampling supplies visual features, masked attention controls visible context, and gated cross-attention enables few-shot multimodal in-context learning.

8 L8 Object Detection

8.1 Task & Metrics

Detection output set $\{(b_i, c_i, s_i)\}_{i=1}^N$: boxes, class, confidence. Harder than classification: variable count/scale/location, overlaps, background. Approaches: sliding window, region proposal, implicit shape/codebook, modern learned queries. **Implicit Shape Model**: local features match codebook entries; each match casts vote for object center/scale, Hough-style detection and rough segmentation. **IoU**: $\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$. Precision/recall: $P = \frac{TP}{TP+FP}$, $R = \frac{TP}{TP+FN}$. TP: is object, exist IoU > threshold. FP: not object, exist IoU > threshold. FN: is object, not exist IoU > threshold.

AP = area under PR curve after sorting detections by confidence; **mAP** = mean over classes/IoU thresholds. Each GT matched once; duplicate boxes become FP. **Anchor/proposal box regression** often uses $t_x = \frac{x-x_a}{w_a}$, $t_y = \frac{y-y_a}{h_a}$, $t_w = \log \frac{w}{w_a}$, $t_h = \log \frac{h}{h_a}$. Detection loss: $\mathcal{L} = \mathcal{L}_{cls} + \lambda \mathcal{L}_{box}$.

8.2 R-CNN Family

R-CNN: Selective Search ~ 2000 proposals, crop/warp each, CNN per proposal, classify + box correction; slow due to repeated convolution. **Fast R-CNN**: one whole-image backbone, crop shared feature with RoIPool, heads. **RoIPool** maps variable RoI to fixed 7×7 by quantized bins; quantization hurts alignment. **Faster R-CNN**: learn proposals with **RPN**. For feature map $H \times W$ and K anchors/location: candidates KHW , objectness logits $2KHW$, box offsets $4KHW$; sort by objectness and keep top ~ 300 proposals. Two-stage = RPN proposals then RoI classify/regress. **Single-stage**: **YOLO/SSD/RetinaNet** dense predict on grid. YOLO divides figure to $S \times S$ patches, predict respectively. output shape: $S \times S \times (5B + C)$, where B boxes has (x, y, w, h) and classes. NMS: keep highest score, remove lower boxes with high IoU, repeat. Final YOLO class-specific score is roughly $P(\text{obj})P(\text{c})$; **cell/grid assignment** makes dense detection fast but can struggle with crowded small objects.

8.3 DETR & Open Vocabulary

DETR: CNN/ViT features + positional encoding + Transformer encoder + learned object queries in decoder; each query predicts class (incl. no-object) and box. **Set prediction** uses **Hungarian matching**: $\hat{\sigma} = \arg \min_{\sigma} \sum_{i=1}^M C(y_i, \hat{y}_{\sigma(i)})$, cost = class + L1 box + GloU. Matching encourages one prediction per GT, reducing NMS need. Tradeoff: clean end-to-end but slow convergence/small objects. **Open-vocabulary detection** replaces fixed class head by text queries. **OWL-ViT**: CLIP-style pretraining then detection fine-tuning; removes CLIP pooling so per-patch/token image embeddings compare to text/image queries, while a box MLP predicts coordinates. Text embedding acts as classifier weights. **OWLv2/self-training** expands detection supervision. **Grounding DINO**: language-aware open-set detector; bridges text phrase to boxes.

9 L9 Dense Prediction

9.1 Segmentation, FCN, Upsampling

Dense prediction: $I \in \mathbb{R}^{H \times W \times 3} \rightarrow Y \in \mathbb{R}^{H \times W \times C}$. **Semantic segmentation** = class per pixel; **instance** = distinguish object instances; **panoptic** = stuff + things. Depth $D \in \mathbb{R}^{H \times W}$; flow $C = 2$; diffusion predicts dense noise/clean image. Core tension: semantic abstraction vs spatial precision. **FCN** converts classifier to fully convolutional: 1×1 conv over feature map gives low-res class score map; bilinear or learned transposed-conv upsampling restores resolution. **Skip fusion** combines high-level semantic coarse maps with low-level fine maps; FCN fuses skip logits by summing. Segmentation metrics: $\text{IoU}_c = \frac{TP_c}{TP_c + FP_c + FN_c}$, $mIoU = \frac{1}{C} \sum_c \text{IoU}_c$. **Upsampling**: interpolation, transposed convolution (linear operator transpose, not inverse), max unpooling with switches, resize+conv to reduce checkerboard. Stride-2 transposed conv can be viewed as inserting zero rows/cols then convolving with flipped kernel; uneven overlap causes checkerboard. **DeconvNet**: encoder-decoder; **U-Net** concatenates encoder features with decoder features and predicts at the end; **FPN**: top-down + lateral connections to semantic pyramid.

9.2 Mask R-CNN, SAM, Depth, Generation

Mask R-CNN = Faster R-CNN + FCN mask head on Rols. Outputs $\{(b_i, c_i, m_i)\}$. **Mask head** $M \in [0, 1]^{K \times m \times m}$, uses per-pixel sigmoid BCE for GT class. **RoIAlign** avoids RoIPool quantization using bilinear sampling; crucial for masks/keypoints. Keypoints as **heatmaps** $\hat{H} \in \mathbb{R}^{K \times H \times W}$. **SAM**: promptable segmentation foundation model. Components: heavy ViT image encoder (cacheable), prompt encoder (points/boxes/text/masks), lightweight mask decoder with two-way attention. Outputs multiple masks for ambiguity + predicted IoU for ranking. Limitations: may miss fine structures, hallucinate small disconnected components, fail boundaries/text-to-mask, and lose to domain-specific

tools. **SA-1B** built with model-in-the-loop stages; **SAM 2** stores previous-frame memory for video, preserving object identity under motion/occlusion and supporting streaming interaction. **Depth Anything**: teacher/student, labeled + 65M unlabeled pseudo labels, DINOv2 backbone; perturbation consistency (color jitter/blur/CutMix) lets student generalize beyond teacher. V2 uses synthetic data for precise depth then bridges sim-to-real. Single-image depth is not true 3D and not necessarily multi-view consistent. **Generation as dense prediction**: GAN/cGAN/Pix2Pix/DDPM/LDM/DiT all predict dense maps over pixels/latents/patches. **PatchGAN** outputs spatial realism map over local patches; **LDM** denoises latent z_t ; **DiT** replaces U-Net denoiser with ViT.

10 L10 Synthetic Data

10.1 Tracking & Pose

Synthetic tracking provides full trajectories and occlusion labels. **Point tracking**: given query point in a frame, predict its location and visibility across time; long-term trajectories and occlusion flags are almost impossible to annotate densely by hand. **Kubric** uses Blender+PyBullet to produce RGB/depth/normal/segmentation/flow/trajectories/visibility. **PIPs**: persistent independent particles track any point; **TAP-Vid-Kubric** benchmark; **TAP-Net/TAPIR** refine per-frame matching into robust point tracking; **CoTracker** tracks points jointly with cross-track attention; **BootsTAPIR/CoTracker3** use teacher-student pseudo labels; **Track-On-R** adds verifier-guided filtering. **6-DoF pose**: model-based uses CAD model/rendering; model-free/few-shot matches RGB-D/reference observations. **FS6D** uses few-shot ShapeNet6D, online augmentation (texture blending/deformation). **MegaPose**: render-and-compare. **FoundationPose**: unified framework trained purely synthetic.

10.2 Depth, Normal, Sensors

Marigold fine-tunes Stable Diffusion for depth; **DAC** generalizes to arbitrary cameras incl. ERP panorama. **Metric depth** is absolute meters; **relative/affine-invariant depth** preserves ordering/shape up to scale/shift. DAC maps arbitrary FoV cameras into an ERP universal canvas via FoV alignment, then maps prediction back. **Camera Depth Models (CDM)** denoise RGB + raw noisy sensor depth into clean metric depth, not monocular depth. **Surface normal**: **Omnidata** uses 3D scans for mid-level cues; normals are ill-posed from images due to shading/texture/lighting ambiguity. **DSINE** injects geometry bias: ray direction encoding from intrinsics, Ray ReLU visible hemisphere, local normal rotation, iterative refinement. **StableNormal** uses diffusion prior with YOSO init + SG-DRN refinement. Key lesson: right inductive bias can replace huge data scale.

11 L11 Curating Vision-Language Data

Alt-text web data: large scale but noisy/mismatched. Uses: generation, VLM, VLA. **Pipeline**: crawl image-text, text filtering, image-text alignment verification, dedup/safety/balancing. **Three schools**: **text-only filtering** (WIT/MetaCLIP), **image-text alignment verification** (CC3M/CC12M/LAION /DataComp/DFN), and **embracing noise/scale** (ALIGN/No Filter/WebLI). Quality controls precision; diversity controls style/long-tail/cultural coverage; safety filtering is still needed even when semantic filtering is relaxed.

11.1 Text-Side Filtering

Text-side filtering: build vocabulary/concepts, match text, balance counts. **WIT** behind CLIP; **WIT vs YFCC**: scale and web alt-text matter. **MetaCLIP** demystifies CLIP data: use metadata vocabulary; cap per concept (e.g. 20k) to balance frequent/rare terms; performance comes from transparent curation + scale.

11.2 Image-Text Alignment

Filtering paradox: stricter filters remove noise but also diversity/long tail. **CC3M**: harvest alt-text, image filtering, text filtering, Cloud Vision API tag overlap, hypernymization; high precision but low recall. **CC12M** keeps image-text verification but relaxes unimodal filters, increasing scale/diversity with lower precision. **LAION** uses CLIP cosine threshold and releases URLs+metadata; democratizes scale but inherits CLIP filter bias and lacks concept balancing. **DataComp**: benchmark dataset curation strategies under fixed compute; top-30% CLIP score is a Western-benchmark sweet spot, not universal optimum; dedup improves generalization. **DFN** trains a dedicated filter on high-quality image-text pairs; filter quality $\approx$ ImageNet accuracy, and poisoned/low-quality filter training data hurts badly. **No Filter/WebLI**: at 10B–100B scale, random noise can dilute but systematic bias/stereotypes amplify; global pretraining + short English fine-tune preserves diversity. Scale helps coverage parity and multilingual/cultural performance, not automatic stereotype correction. Datasets still need NSFW/toxicity/PII/watermark/low-quality filtering, but overfiltering harms coverage.

12 L12 Diffusion & Score Models

12.1 EBM, DCM

Density modeling wants sample from $p(x)$. **EBM**: $p_{\theta}(x) = \exp(-E_{\theta}(x))/Z_{\theta}$, Z intractable. **Langevin dynamics** uses score: $x_{k+1} = x_k + \frac{\eta}{2} \nabla_x \log p(x_k) + \sqrt{\eta} \epsilon_k$. **Score matching** learns $s_{\theta}(x) = \nabla_x \log p_{\text{data}}(x)$ without Z : $\mathcal{J}_{\text{SM}} = \frac{1}{2} \mathbb{E}_{p_{\text{data}}} \|s_{\theta}(x) - \nabla_x \log p_{\text{data}}(x)\|^2$, $\min_{\theta} \mathcal{J}_{\text{SM}} \equiv \min_{\theta} \mathbb{E}_{p_{\text{data}}} \left[\frac{1}{2} \|s_{\theta}(x)\|^2 + \nabla_x \cdot s_{\theta}(x) \right]$. **Hyvarinen score matching** removes unknown $\nabla \log p$ by integration by parts; **denoising score matching** avoids second derivatives by learning scores of noisy conditional Gaussians. With $x_{\sigma} = x + \sigma \epsilon$: $\nabla_{x_{\sigma}} \log p_{\sigma}(x_{\sigma}) = -(x_{\sigma} - x)/\sigma^2$. **Noise Conditional Score Network (NCSN)** learns $s_{\theta}(x, \sigma) \approx \nabla_x \log p_{\sigma}(x)$ for multiple noise levels $\sigma_1 > \dots > \sigma_L$.

Annealed Langevin Dynamics samples coarse-to-fine; for each σ_i , run $x_{k+1} = x_k + \frac{\eta_i}{2} s_{\theta}(x_k, \sigma_i) + \sqrt{\eta_i} \epsilon_k$. Metropolis-Hastings accepts/rejects random-walk proposals but mixes slowly in high dimensions;

Langevin uses score direction for informed updates.

12.2 DDPM

Forward: $q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, (1-\alpha_t)I)$, $\alpha_t = 1 - \beta_t$, $\tilde{\alpha}_t = \prod_{s=1}^t \alpha_s$, $q(x_t|x_0) = \mathcal{N}(\sqrt{\tilde{\alpha}_t}x_0, (1-\tilde{\alpha}_t)I)$, $x_t = \sqrt{\tilde{\alpha}_t}x_0 + \sqrt{1-\tilde{\alpha}_t}\epsilon$. **Reverse**: $p_{\theta}(x_{t-1}|x_t) = \mathcal{N}(\mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$.

Posterior: $q(x_{t-1}|x_t, x_0) = \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t I)$, $\tilde{\mu}_t = \frac{\sqrt{\alpha_{t-1}}\tilde{\beta}_t}{1-\tilde{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1-\tilde{\alpha}_{t-1})}{1-\tilde{\alpha}_t}x_t$, $\tilde{\beta}_t = \frac{1-\tilde{\alpha}_t-1}{1-\tilde{\alpha}_t}\beta_t$.

Noise parameterization: $x_0 = \frac{x_t - \sqrt{1-\tilde{\alpha}_t}\epsilon}{\sqrt{\tilde{\alpha}_t}}$, $\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \|\epsilon - \epsilon_{\theta}(x_t, t)\|_2^2$.

Sampling: $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\tilde{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$.

12.3 Continuous View & Equivalent Targets

SDE/PF-ODE $dx = f(x, t) dt + g(t) dw$, $dx = \left[f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x) \right] dt$. unify score/diffusion.

Gaussian path: $z_t = \alpha_t x + \sigma_t \epsilon$, $\text{SNR} = \alpha_t^2 / \sigma_t^2$, $\lambda = \log \text{SNR}$.

Equivalent targets: clean x , noise ϵ , score $s_t(z) = -\mathbb{E}[\epsilon|z_t = z]/\sigma_t$, velocity/path derivative $v_t = \dot{\alpha}_t x + \dot{\sigma}_t \epsilon$, denoiser $D(z_t) = \mathbb{E}[x|z_t]$. **Weighting and schedule** can be expressed in SNR/log-SNR coordinates; epsilon-prediction is not uniform in SNR, log-SNR makes interpretation cleaner. **Forward paths**: VE/EDM $x_t = x_0 + \sigma_t \epsilon$; VP/DDPM $x_t = \alpha_t x_0 + \sigma_t \epsilon$, $\alpha_t^2 + \sigma_t^2 = 1$; Flow Matching $x_t = (1-t)x_0 + t\epsilon$. Only VP keeps variance normalized. **VLB weight**: in x_0 prediction, DDPM VLB weights each step by SNR drop, $L_{t-1} \propto (t_{t-1} - t_t) \|D_{\theta}(x_t, t) - x_0\|^2$, $r = \text{SNR}$. **Training design**: choose path, predictive target (ϵ, x_0, s, v) , objective weight $w(\lambda)$, and noise proposal/schedule $q(\lambda)$; Monte Carlo estimator uses weight w/q . Slides compare uniform, EDM/sigmoid, and Min-SNR weighting; monotone weighted objectives can be rewritten as mixtures of ELBOs on Gaussian-noise-augmented data.

13 L13 Beyond Diffusion

13.1 Normalizing Flow

Normalizing flow uses invertible $z = f_{\theta}(x)$ with **exact likelihood**: $\log p_X(x) = \log p_Z(f_{\theta}(x)) + \log \left| \det \frac{\partial f_{\theta}(x)}{\partial x} \right|$. **MLE** minimizes negative log-likelihood. **Composition**: log-det sums over layers. Bottleneck: invertibility + tractable Jacobian determinant, dimension match. **Triangular Jacobian** makes determinant product of diagonals. **NICE additive coupling**: $y_a = x_a$, $y_b = x_b + m(x_a)$, $\det J = 1$. **RealNVP affine coupling**: $y_a = x_a$, $y_b = x_b \odot \exp(s(x_a)) + t(x_a)$, $\log |\det J| = \sum s(x_a)$. **TarFlow**: transformer masked autoregressive flow over patches; triangular Jacobian via ordering. **Autoregressive affine form**: $z_i = \frac{x_i - \mu_i(x_{<i})}{\sigma_i(x_{<i})}$, $x_i = \mu_i(x_{<i}) + \sigma_i(x_{<i})z_i$, $\log |\det J| = \sum_i \log \sigma_i$. **Noise augmentation**: helps OOD generalization; without noise, $f^{-1}(z)$ only sees neighborhoods of discrete training samples, while sampling latent Gaussian maps through continuous space. **Tweedie denoising**: $\mathbb{E}[x|y] = y + \sigma^2 \nabla_y \log p(y)$. **Guidance**: analogous classifier-free extrapolation between conditional and null parameters, $\tilde{\mu}_i^c = (1+w)\mu_i^c(x) - w\mu_i^c(\text{null})$, $\tilde{\alpha}_i^c = (1+w)\alpha_i^c(x) - w\alpha_i^c(\text{null})$. **Unconditional guidance**: ramp strength over order index, $\tilde{\mu}_i^u = (1+w_i)\mu_i^u(1) - w_i\mu_i^u(\tau)$, $w_i = \frac{1}{T-i}w$.

13.2 GAN, AR, Masked/Flexible-Order

GAN objective: $\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} \log D(x) + \mathbb{E}_{z \sim p_Z} \log(1 - D(G(z)))$. Pros: fast high-fidelity samples; cons: unstable, mode collapse, no exact likelihood. **Optimal discriminator**: $D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}, V(D^*, G) = -\log 4 + 2 \text{JSD}(p_{\text{data}} \| p_G)$. If supports barely overlap, **JS saturation** makes generator gradients poor; with $D(G(z)) = \sigma(f)$, the $\log(1-D)$ term can cause early **gradient saturation**. **WGAN**: replaces JS by Earth-Mover/Wasserstein-1, $W(p_{\text{data}}, p_G) = \inf_{\gamma \in \Pi(p_{\text{data}}, p_G)} \mathbb{E}_{(x, y) \sim \gamma} \|x - y\| = \sup_{f \|f\|_L \leq 1} \mathbb{E}_{x \sim p_{\text{data}}} f(x) - \mathbb{E}_{x \sim p_G} f(x)$, with 1-Lipschitz critic, giving useful gradients even for non-overlapping supports. **RpGAN**: compares paired real/fake scores, $\mathbb{E}_{z, x} [f(D(G(z)) - D(x))]$; each fake competes with real samples, so single-mode collapse still loses to other real modes. **R3GAN**: adds principled R1/R2-style regularization to stabilize dynamics. **ADD**: uses adversarial loss for diffusion distillation, $\mathcal{L}_{\text{student}} = \mathcal{L}_{\text{distill}} + \lambda \mathcal{L}_{\text{adv}}$, where $\mathcal{L}_{\text{distill}}$ comes from diffusion teacher. AR factorizes exact likelihood: $p(x) = \prod_{i=1}^N p(x_i|x_{<i})$, stable MLE/cross-entropy but serial $O(N)$ sampling and ordering bias. Image AR tokenizes pixels/patches/codebook tokens; Transformer provides long-range context. **AR evolution**: **RIDE** is a raw-pixel density estimator with spatial LSTMs, but recurrent state forces serial training/sampling. **PixelRNN** also models raw pixels sequentially; **PixelCNN** uses masked convolution so training is parallel over pixels but sampling remains sequential. **VQ-VAE** shortens image sequence via discrete codebook tokens. Loss has reconstruction + codebook + commitment terms; straight-through estimator copies gradient through nearest-code lookup. **JetFormer** soft tokens use invertible flow tokenizer + AR Transformer, preserving continuous information and exact likelihood more directly than hard codebooks. Masked/flexible-order generation predicts masked tokens in iterative refinement; faster than AR, useful for editing/painting, likelihood often approximate or schedule-dependent. **Masked AR**: **MaskGIT** uses VQGAN tokenizer + bidirectional Transformer; train by masking random token subset and cross-entropy on masked tokens only. Iterative decoding predicts all masked positions in parallel, samples tokens, keeps high-confidence ones, and follows a mask schedule; steps drop from $O(N)$ to about 8–12, but bidirectional attention cannot use KV cache. **RandAR**: random-order causal generation adds position instruction tokens, allowing KV-cache and parallel decoding of multiple predicted positions while avoiding fixed raster order.